CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
CAMPUS TIMÓTEO

Arthur Baltar Eler

ALGORITMOS E ESTRUTURAS DE DADOS II: ÁRVORES, BUSCA EM LARGURA E QUADTREE

Timóteo

2024

Arthur Baltar Eler

ALGORITMOS E ESTRUTURAS DE DADOS II: ÁRVORES, BUSCA EM LARGURA E QUADTREE

Relatório elaborado para a disciplina de Algoritmos e Estruturas de Dados II, como parte do curso de Engenharia de Computação do Campus Timóteo do Centro Federal de Educação Tecnológica de Minas Gerais. Este trabalho tem como objetivo elucidar os conceitos de árvores, Busca em Largura (BFS) e Quadtree na computação. A instituição, referida aqui como CEFET-MG, é reconhecida pela excelência em educação tecnológica.

Orientador: Leonardo Lacerda Alves

Timóteo

2024

.

.

.

Dedico a
todos aqueles que buscam incessantemente aprender.

Agradecimentos

Agradeço, primeiramente, a Deus.

Também agradeço a todos aqueles que, de alguma forma, contribuíram para o meu crescimento pessoal e profissional, e para a concretização deste trabalho.

“Sapere aude”.
Immanuel Kant

Resumo

Este trabalho oferece uma explanação detalhada sobre os conceitos de árvores, Busca em Largura (BFS) e Quadtree na computação. Aborda desde a estrutura hierárquica das árvores até suas aplicações em estruturas de dados, seguido por exemplos práticos de codificação, incluindo técnicas de inserção fora de ordem, exclusão de elementos no meio da árvore, e métodos de impressão de valores por iteração na árvore. Também são detalhados os algoritmos utilizados para BFS e Quadtree, incluindo sua implementação prática e explicações passo a passo. Além disso, são apresentados os prós e contras da utilização dessas estruturas, destacando suas vantagens, como a eficiência em operações de busca e ordenação no caso das árvores, e a eficiência espacial no caso das Quadtrees, bem como os desafios, como a complexidade de implementação e a necessidade de balanceamento em algumas situações. Este relatório oferece uma visão abrangente e prática do uso de árvores, BFS e Quadtree na computação, com uma base teórica sólida e exemplos concretos de suas aplicações em algoritmos e estruturas de dados.

Palavras-chave: árvores, Busca em Largura, BFS, Quadtree, estrutura de dados, codificação, inserção, exclusão, impressão, iteração.

Abstract

This work provides a detailed explanation of the concepts of trees, Breadth-First Search (BFS), and Quadtree in computing. It covers the hierarchical structure of trees and their applications in data structures, followed by practical coding examples, including techniques for out-of-order insertion, deletion of elements in the middle of the tree, and methods for printing values iteratively in the tree. The algorithms used for BFS and Quadtree are also detailed, including their practical implementation and clear, step-by-step explanations. Additionally, the pros and cons of using these structures are presented, highlighting their advantages, such as efficiency in search and sorting operations for trees and spatial efficiency for Quadtrees, as well as challenges such as implementation complexity and the need for balancing in some situations. This report offers a comprehensive and practical view of the use of trees, BFS, and Quadtree in computing, with a solid theoretical foundation and concrete examples of their application in algorithms and data structures.

Keywords: trees, Breadth-First Search, BFS, Quadtree, data structures, coding, insertion, deletion, printing, iteration.

Lista de ilustrações

Figura 1 – Árvore Binária 10
Figura 2 – Algoritmo BFS percorrendo uma árvore 21
Figura 3 – Quadtree em jogos 26

Listings

1.1 Classe Principal 11
1.2 Inserir 12
1.3 Remover 13
1.4 Imprimir 16
2.1 Implementação de BFS em Java 22
3.1 Implementação de uma Quadtree em Java 27

Sumário

Listings **8**

1 CAPÍTULO 1 . . . . . **10**
1.1 O que é uma árvore? **10**
1.2 Codificação **11**
1.3 Explicação **17**
1.4 Prós e Contras **19**

2 CAPÍTULO 2 . . . . . **21**
2.1 Definição **21**
2.2 Características **21**
2.3 Codificação **22**
2.3.1 Explicação do Código 23
2.4 Busca em Grafos Ponderados: **24**

3 CAPÍTULO 3 . . . . . **25**
3.1 Definição **25**
3.2 Aplicação em Consultas Espaciais **25**
3.3 Complexidade da Quadtree **25**
3.3.1 Complexidade de Tempo 25
3.3.2 Complexidade de Espaço 26
3.3.3 Considerações Adicionais 26
3.4 Exemplo Prático: Detecção de Colisão em Jogos **26**
3.5 Codificação **27**
3.5.1 Explicação do Código 31
3.6 Vantagem e Desvantagem **31**
3.6.1 Espaços não uniformes 31
3.6.2 Inserção e Remoção 31

REFERÊNCIAS **32**

1 Árvores

“Sem o respaldo do desempenho e do trabalho duro,
as palavras não significam nada”.
Michael Jordan

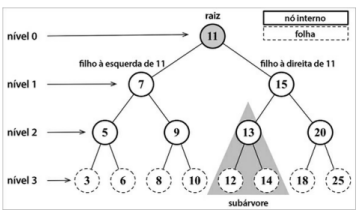
Os inúmeros métodos de estruturação de dados, já familiares ao leitor, tais como listas encadeadas, filas e pilhas, são aplicados para resolver uma ampla variedade de problemas do mundo real. No entanto, à medida que a complexidade desses problemas se intensifica, é observável que cada uma dessas estruturas apresenta desempenhos variados em cenários particulares. Como consequência, os engenheiros de computação e outros pesquisadores da área de tecnologia tendem a buscar alternativas mais sofisticadas à medida que a complexidade dos desafios aumenta. Um exemplo paradigmático de tal sofisticação é a árvore. Neste primeiro capítulo, nossa intenção é investigar esta estrutura de dados peculiar, delineando suas propriedades distintivas e oferecendo ilustrações de implementação.

1.1 O que é uma árvore?

Uma árvore é um tipo de estrutura de dados hierárquica que se caracteriza por sua organização ascendente, contendo elementos designados como "nós". Cada nó, à exceção de um, é associado a um nó pai e pode conter zero ou mais nós filhos. Esta exceção, denominada nó raiz, está situada no ápice da estrutura, ocupando a posição de maior elevação e, portanto, desprovida de um nó pai. (GOODRICH; TAMASSIA, 2013)

Esta estrutura, de caráter ascendente, tem seus elementos, também referidos como vértices, como representações das informações a serem processadas. As inter-relações entre esses nós são estabelecidas através de arestas, que podem ser unidirecionais (limitadas a uma única direção entre nós, sem retorno pelo mesmo trajeto) ou bidirecionais (permitindo a transição entre nós sem restrições). É pertinente observar que cada nó filho está associado a apenas um nó pai, enquanto um nó pai pode ter múltiplos nós filhos, de acordo com as especificações do tipo de árvore em questão. (RICARDO, 2015)

Figura 1 – Árvore Binária



Fonte: Estruturas de Dados e Algoritmos com JavaScript - Loiane Groner

Na figura 3, é apresentado um exemplo ilustrativo de uma árvore. Conforme discutido por (GRONER, 2019), os nós podem ser categorizados como internos ou externos. Os nós internos, por definição, possuem ao menos um filho (exemplificados por 7, 5, 9, 15, 13 e 20). Em contrapartida, os nós externos, também denominados folhas, carecem de filhos diretos (exemplificados por 3, 6, 8, 10, 12, 14, 18 e 25).

Cada nó pode ser contextualizado dentro de uma árvore genealógica, possuindo ancestrais e descendentes. Os ancestrais de um nó, excluindo a raiz, são seus predecessores diretos, como pai, avô, bisavô, e assim por diante. Os descendentes de um nó são suas posteridades diretas, como filhos, netos, bisnetos, e assim sucessivamente. Por exemplo, o nó 5 possui 7 e 11 como ancestrais, e 3 e 6 como descendentes, como explicitado por (GRONER, 2019), ilustrando a hierarquia inerente aos nós.

Ademais, algumas características adicionais das árvores incluem:

1. Subárvores: constituídas por um nó e seus descendentes imediatos. Por exemplo, os nós 13, 12 e 14 formam uma subárvore na estrutura ilustrada na figura 3.
2. Profundidade: corresponde ao número de níveis hierárquicos acima de um nó específico. Por exemplo, o nó 3 tem profundidade igual a 3 devido aos três níveis hierárquicos acima dele (5, 7 e 11).
3. Altura e níveis: a altura de uma árvore é o número máximo de níveis hierárquicos, medida do nível da raiz ao nível da folha mais distante. A raiz está situada no nível 0, seus filhos no nível um, e assim por diante.
4. Árvores são acíclicas e conectadas, significando que não contêm ciclos e todos os nós estão ligados de alguma forma.

1.2 Codificação

Para uma elucidativa compreensão dos conceitos previamente discutidos, apresenta-se um exemplo prático de implementação de uma árvore binária, acompanhado das operações de inserção, remoção e impressão:

1. Classe *ArvoreBinariaApp*:

```
1  class ArvoreBinariaApp {
2    public static void main(String[] args) {
3      Scanner le = new Scanner(System.in);
4      Tree arv = new Tree();
5      int opcao;
6      long x;
7      System.out.print("\n Programa Arvore binaria de long");
8      do {
9          System.out.print("\n*************************************");
10         System.out.print("\nEntre com a opcao:");
```

```
11      System.out.print("\n ----1: Inserir");
12      System.out.print("\n ----2: Excluir");
13      System.out.print("\n ----3: Pesquisar");
14      System.out.print("\n ----4: Exibir");
15      System.out.print("\n ----5: Sair do programa");
16      System.out.print("\n*********************************");
17      System.out.print("\n-> ");
18      opcao = le.nextInt();
19      switch(opcao) {
20          case 1: {
21                  System.out.print("\n Informe o valor (long) -> ");
22                  x = le.nextLong();
23                  arv.inserir(x);
24                  break;
25          }
26          case 2: {
27                  System.out.print("\n Informe o valor (long) -> ");
28                  x = le.nextLong();
29                  if ( !arv.remover(x) )
30                              System.out.print("\n Valor nao encontrado
                                !");;
31                  break;
32          }
33          case 3: {
34                  System.out.print("\n Informe o valor (long) -> ");
35                          x = le.nextLong();
36                      if( arv.buscar(x) != null )
37                         System.out.print("\n Valor Encontrado");
38                  else
39                     System.out.print("\n Valor nao encontrado!");
40                  break;
41          }
42          case 4: {
43                 arv.caminhar();
44                 break;
45          }
46          } // fim switch
47      } while(opcao != 5);
48
49    }
50  }
```

Listing 1.1 – Classe Principal

2. Método *Inserir*:

```
1   class Tree {
2     private No root; // raiz
3
4     public Tree() { root=null; } // inicializa arvore
```

```
5
6   public void inserir(long v) {
7     No novo = new No(); // cria um novo NÃş
8     novo.item = v; // atribui o valor recebido ao item de dados do
          NÃş
9     novo.dir = null;
10    novo.esq = null;
11
12    if (root == null) root = novo;
13    else  { // se nao for a raiz
14      No atual = root;
15      No anterior;
16      while(true) {
17        anterior = atual;
18        if (v <= atual.item) { // ir para esquerda
19          atual = atual.esq;
20          if (atual == null) {
21            anterior.esq = novo;
22            return;
23          }
24        }  // fim da condiÃğÃčo ir a esquerda
25        else { // ir para direita
26           atual = atual.dir;
27           if (atual == null) {
28             anterior.dir = novo;
29             return;
30           }
31        } // fim da condiÃğÃčo ir a direita
32      } // fim do laÃğo while
33    } // fim do else nÃčo raiz
34
35  }
```

Listing 1.2 – Inserir

3. Método *remover*:

```
1  public boolean remover(long v) {
2      if (root == null) return false; // se arvore vazia
3
4      No atual = root;
5      No pai = root;
6      boolean filho_esq = true;
7
8      // ****** Buscando o valor **********
9      while (atual.item != v) { // enquanto nao encontrou
10       pai = atual;
11       if(v < atual.item ) { // caminha para esquerda
12         atual = atual.esq;
13         filho_esq = true; // ÃĽ filho a esquerda? sim
```

```
14        }
15        else { // caminha para direita
16          atual = atual.dir;
17          filho_esq = false; // ÃĽ filho a esquerda? NAO
18        }
19        if (atual == null) return false; // encontrou uma folha ->
              sai
20      } // fim laÃğo while de busca do valor
21
22      // ************************************************
23      // se chegou aqui quer dizer que encontrou o valor (v)
24      // "atual": contem a referencia ao No a ser eliminado
25      // "pai": contem a referencia para o pai do No a ser eliminado
26      // "filho_esq": ÃĽ verdadeiro se atual ÃĽ filho a esquerda do
            pai
27      // ************************************************
28
29      // Se nao possui nenhum filho (ÃĽ uma folha), elimine-o
30      if (atual.esq == null && atual.dir == null) {
31        if (atual == root ) root = null; // se raiz
32        else if (filho_esq) pai.esq = null; // se for filho a
              esquerda do pai
33          else pai.dir = null; // se for filho a direita do pai
34      }
35
36      // Se ÃĽ pai e nao possui um filho a direita, substitui pela
            subarvore a direita
37      else if (atual.dir == null) {
38         if (atual == root) root = atual.esq; // se raiz
39         else if (filho_esq) pai.esq = atual.esq; // se for filho a
               esquerda do pai
40           else pai.dir = atual.esq; // se for filho a direita do
                 pai
41      }
42
43      // Se ÃĽ pai e nao possui um filho a esquerda, substitui pela
            subarvore a esquerda
44      else if (atual.esq == null) {
45         if (atual == root) root = atual.dir; // se raiz
46         else if (filho_esq) pai.esq = atual.dir; // se for filho a
               esquerda do pai
47           else pai.dir = atual.dir; // se for  filho a direita do
                 pai
48      }
49
50      // Se possui mais de um filho, se for um avÃť ou outro grau
            maior de parentesco
51      else {
```

```
52        No sucessor = no_sucessor(atual);
53        // Usando sucessor que seria o NÃş mais a esquerda da
              subarvore a direita do No que deseja-se remover
54        if (atual == root) root = sucessor; // se raiz
55        else if(filho_esq) pai.esq = sucessor; // se for filho a
              esquerda do pai
56          else pai.dir = sucessor; // se for filho a direita do
                pai
57        sucessor.esq = atual.esq; // acertando o ponteiro a esquerda
              do sucessor agora que ele assumiu
58                                  // a posiÃğÃčo correta na arvore
59      }
60
61      return true;
62    }
63
64    // O sucessor Ã Ľ o NÃş mais a esquerda da subarvore a direita do
          No que foi passado como parametro do metodo
65    public No no_sucessor(No apaga) { // O parametro Ã Ľ a referencia
          para o No que deseja-se apagar
66      No paidosucessor = apaga;
67      No sucessor = apaga;
68      No atual = apaga.dir; // vai para a subarvore a direita
69
70      while (atual != null) { // enquanto nao chegar no NÃş mais a
            esquerda
71        paidosucessor = sucessor;
72        sucessor = atual;
73        atual = atual.esq; // caminha para a esquerda
74      }
75      // ********************************************************
76      // quando sair do while "sucessor" serÃą o NÃş mais a esquerda
             da subarvore a direita
77      // "paidosucessor" serÃą o o pai de sucessor e "apaga" o NÃş
            que deverÃą ser eliminado
78      // ********************************************************
79      if (sucessor != apaga.dir) { // se sucessor nao Ã Ľ o filho a
            direita do NÃş que deverÃą ser eliminado
80        paidosucessor.esq = sucessor.dir; // pai herda os filhos do
              sucessor que sempre serÃčo a direita
81        // lembrando que o sucessor nunca poderÃą ter filhos a
              esquerda, pois, ele sempre serÃą o
82        // NÃş mais a esquerda da subarvore a direita do NÃş apaga.
83        // lembrando tambÃĽm que sucessor sempre serÃą o filho a
              esquerda do pai
84
85        sucessor.dir = apaga.dir; // guardando a referencia a
              direita do sucessor para
```

```
86                                      // quando ele assumir a posiÃğÃčo
                                           correta na arvore
87      }
88      return sucessor;
89    }
```

Listing 1.3 – Remover

4. Método *imprimir*:

```
1  // Non-recursive java program for inorder traversal
2
3  import java.util.Stack;
4
5  // Class containing left and right child of
6  // current node and key value
7  class Node
8  {
9      int data;
10     Node left, right;
11
12     public Node(int item)
13     {
14         data = item;
15         left = right = null;
16     }
17 }
18
19 // Class to print the inorder traversal
20 class BinaryTree
21 {
22     Node root;
23     void inorder()
24     {
25         if (root == null)
26             return;
27
28
29         Stack<Node> s = new Stack<Node>();
30         Node curr = root;
31
32         // Traverse the tree
33         while (curr != null || s.size() > 0)
34         {
35
36             // Reach the left most Node of the
37             // curr Node
38             while (curr !=  null)
39             {
40                 // Place pointer to a tree node on
```

```
41                  // the stack before traversing
42                  // the node's left subtree
43                  s.push(curr);
44                  curr = curr.left;
45              }
46
47              // Current must be NULL at this point
48              curr = s.pop();
49
50              System.out.print(curr.data + " ");
51
52              // we have visited the node and its
53              // left subtree. Now, it's right
54              // subtree's turn
55              curr = curr.right;
56          }
57      }
58
59      public static void main(String args[])
60      {
61          // Creating a binary tree and entering
62          // the nodes
63          BinaryTree tree = new BinaryTree();
64          tree.root = new Node(1);
65          tree.root.left = new Node(2);
66          tree.root.right = new Node(3);
67          tree.root.left.left = new Node(4);
68          tree.root.left.right = new Node(5);
69          tree.inorder();
70      }
71  }
```

Listing 1.4 – Imprimir

A estrutura de dados examinada anteriormente é conhecida como "Árvore Binária em Java"(ArvoreBinariaApp). Este algoritmo foi concebido por (BARBOSA, 2017) e disponibilizado no repositório GitHub, com o propósito de facilitar a criação de uma árvore binária em Java. Tanto as operações de inserção quanto de remoção são parte integrante dessa implementação. Contudo, o método de impressão adotado é um fragmento de código proveniente do site GeeksforGeeks, conforme citado por (VATSA, 2023).

1.3 Explicação

A classe "ArvoreBinariaApp"é o epicentro que abriga a função 'main', onde o usuário pode eleger qual operação deseja executar.

A inclusão de elementos na árvore nesta implementação segue os princípios de uma Árvore Binária de Busca (BST - Binary Search Tree):

1. Criação de um novo nó: Um novo nó é instanciado para armazenar o valor a ser inserido na árvore. O valor em questão é atribuído ao item de dados do novo nó.

2. Verificação da raiz: É averiguado se a raiz da árvore é nula. Se for, indica que a árvore está vazia, e o novo nó torna-se a raiz.

3. Inserção na árvore não vazia: Caso a raiz não seja nula, o novo valor é inserido na posição adequada na árvore. A busca se inicia a partir da raiz, comparando o valor a ser inserido com o valor do nó atual. Se o valor for menor ou igual ao valor do nó atual, o algoritmo desce pela subárvore esquerda. Se o valor for maior que o valor do nó atual, o algoritmo desce pela subárvore direita.

4. Deslocamento na árvore: O algoritmo prossegue descendo na árvore até encontrar um nó onde a subárvore correspondente seja nula. Nesse ponto, o novo nó é inserido como filho esquerdo ou direito do último nó visitado, dependendo da comparação realizada.

5. Finalização da inserção: Após a inclusão do novo nó, o processo é concluído, e a árvore mantém-se válida como uma BST.

A remoção de elementos na árvore neste contexto segue os princípios de uma Árvore Binária de Busca (BST - Binary Search Tree):

1. Verificação da árvore vazia: O procedimento inicia-se com a verificação da existência de uma árvore vazia, examinando se a raiz é nula. Se a árvore estiver vazia, o método retorna false, indicando que o valor não pode ser removido por não estar presente na árvore.

2. Busca pelo valor a ser removido: O algoritmo conduz uma busca pelo valor a ser removido na árvore, aderindo aos princípios de uma Árvore Binária de Busca. Durante essa busca, o método mantém um ponteiro para o nó atual, um ponteiro para o pai do nó atual e uma variável booleana para indicar se o nó atual é filho esquerdo ou direito do pai.

3. Identificação do nó a ser removido: Após localizar o valor a ser removido na árvore, o método identifica o nó a ser removido, juntamente com seu pai e determina se ele é filho esquerdo ou direito do pai.

4. Casos de remoção: O procedimento lida com três cenários distintos de remoção, dependendo das características do nó a ser removido: Se o nó a ser removido não possui nenhum filho (é uma folha), ele é removido simplesmente desvinculando-o do seu pai. Se o nó a ser removido possui apenas um filho (esquerdo ou direito), ele é substituído pelo seu filho. Se o nó a ser removido tem dois filhos, ele é substituído pelo seu sucessor na árvore (o nó mais à esquerda na subárvore direita), e o sucessor ocupa sua posição na árvore.

5. Método do sucessor: Este método localiza o sucessor de um nó que será removido da árvore, retornando o nó mais à esquerda na subárvore direita do nó que está sendo

removido. Durante essa busca, o método mantém referências para o nó sucessor, seu pai e o nó que está sendo removido.

6. Finalização da remoção: Após a remoção do nó, o método retorna true, indicando que o valor foi removido com sucesso da árvore.

Esse processo de remoção assegura que a Árvore Binária de Busca permaneça válida e conserve suas propriedades após a remoção de um valor. Ele preserva a ordem dos elementos na árvore e garante a integridade estrutural da mesma.

Por último, temos o método de impressão, o qual foi implementado de maneira iterativa. O método 'inorder' na classe BinaryTree realiza a impressão da travessia em ordem (inorder traversal) de uma árvore binária. Esta abordagem emprega uma pilha para efetuar a travessia em ordem de uma árvore binária. O processo tem início na raiz, percorre a subárvore esquerda até o limite máximo, imprime os nós ao longo deste percurso, e então direciona-se para a subárvore direita. Esta metodologia assegura que os nós sejam impressos em ordem crescente.

1.4 Prós e Contras

Existem diversas abordagens para a implementação dos métodos de inserção, remoção e impressão em uma árvore binária, logo, é pertinente uma análise crítica sobre a forma como esses algoritmos foram desenvolvidos, com o intuito de extrair aprendizados valiosos.

1. Inserção: Os pontos positivos deste algoritmo residem no seu aderência ao princípio de uma árvore binária de busca, assegurando a inserção ordenada dos elementos, o que facilita operações de busca subsequentes. Além disso, a abordagem é implementada de forma iterativa, potencialmente mais eficiente em termos de consumo de memória e tempo de execução, comparativamente às abordagens recursivas.

   Por outro lado, as limitações incluem a ausência de qualquer mecanismo de balanceamento da árvore durante a inserção, o que pode resultar em árvores desbalanceadas e degeneradas em casos extremos, impactando negativamente o desempenho de operações de busca e outras operações.

2. Remoção: Este algoritmo destaca-se por tratar adequadamente os diferentes cenários de remoção, incluindo a remoção de nós com zero, um ou dois filhos, enquanto mantém a propriedade fundamental da árvore binária de busca, preservando sua ordenação.

   A desvantagem potencial reside na possibilidade de desempenho inadequado em grandes árvores, especialmente quando desbalanceadas, uma vez que a remoção pode exigir ajustes extensivos na estrutura da árvore.

3. Impressão: As vantagens desta técnica de impressão incluem a disposição dos elementos da árvore em ordem crescente, o que é benéfico em muitos contextos de aplicação. Além disso, o algoritmo adota uma abordagem iterativa e faz uso de uma pilha, potencialmente mais eficiente em termos de consumo de memória e tempo de execução.

No entanto, uma desvantagem considerável é a complexidade para aqueles não familiarizados com a travessia em ordem de árvores binárias utilizando uma pilha, o que pode dificultar a compreensão do código e a identificação de possíveis erros.

Ao término dessa análise, é evidente que os algoritmos fornecidos são eficazes para operações básicas em uma árvore binária de busca, mas podem não ser otimizados para todos os cenários, especialmente em relação ao balanceamento da árvore e ao desempenho das operações de remoção em árvores grandes e desbalanceadas.

Portanto, cabe ao leitor compreender o problema em questão, avaliar se uma árvore binária é a solução mais adequada e, em seguida, buscar as melhores práticas de implementação, uma vez que não há uma abordagem única que seja superior, mas sim diferentes formas que atendem a diferentes necessidades.

2 Breadth-first search

“No meio da dificuldade encontra-se a oportunidade. ”
Albert Einstein

2.1 Definição

Breadth-First Search (BFS) é um algoritmo fundamental em ciência da computação utilizado para percorrer ou buscar em grafos ou árvores.

O BFS começa em um nó raiz (ou outro nó arbitrário de um grafo), explora todos os seus vizinhos antes de mover-se para os vizinhos de seus vizinhos. Isso garante que o BFS visite todos os nós em um nível antes de descer para o próximo nível (CORMEN et al., 2009).

A estrutura de dados fundamental utilizada é a fila (queue). A fila é uma estrutura de dados que segue a política FIFO (First In, First Out), onde os elementos são inseridos no final e removidos do início. Essa característica é crucial para o funcionamento do BFS, pois permite que os nós sejam processados na ordem em que são descobertos. Ao iniciar a busca a partir de um nó inicial, o BFS coloca esse nó na fila e, em seguida, processa todos os seus vizinhos, adicionando-os à fila conforme são descobertos. A fila garante que todos os nós no mesmo nível de profundidade sejam visitados antes de passar para o próximo nível, o que é fundamental para garantir que o algoritmo explore o grafo em camadas.

Figura 2 – Algoritmo BFS percorrendo uma árvore

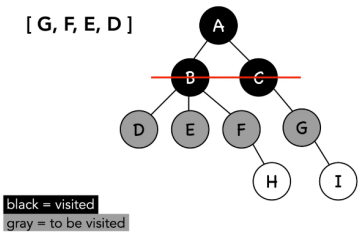


Imagem proveniente (SAMBOL, 2022)

2.2 Características

Algumas características importantes do BFS incluem:

- **Completude**: O BFS encontrará a solução se houver uma (CORMEN et al., 2009).
- **Detecção de Ciclo**: Pode ser usado para detectar ciclos em um grafo não direcionado (SEDGEWICK; WAYNE, 2011).
- **Componentes Conexos**: Ajuda na identificação de componentes conexos em grafos não direcionados (CORMEN et al., 2009).

- **Otimização**: Se todos os passos têm o mesmo custo, o BFS encontrará a solução mais curta (SEDGEWICK; WAYNE, 2011).
- **Complexidade de Tempo**: A complexidade de tempo do BFS é O(V + E), onde V é o número de vértices e E é o número de arestas (SEDGEWICK; WAYNE, 2011).
- **Complexidade de Espaço**: O BFS requer memória proporcional ao número de vértices, resultando em uma complexidade de espaço de O(V) (CORMEN et al., 2009).

2.3 Codificação

Para uma melhor compreensão dos conceitos previamente discutidos, apresenta-se um exemplo prático de implementação de uma busca em largura (BFS) em Java:

```
1  import java.util.ArrayList;
2  import java.util.LinkedList;
3  import java.util.List;
4  import java.util.Queue;
5
6  public class GfG {
7
8      // BFS from given source s
9      static void bfs(List<List<Integer>> adj, int s,
10                              boolean[] visited) {
11
12         // Create a queue for BFS
13         Queue<Integer> q = new LinkedList<>();
14
15         // Mark the source node as visited and
16         // enqueue it
17         visited[s] = true;
18         q.offer(s);
19
20         // Iterate over the queue
21         while (!q.isEmpty()) {
22
23             // Dequeue a vertex from queue and print it
24             int curr = q.poll();
25             System.out.print(curr + " ");
26
27             // Get all adjacent vertices of the dequeued
28             // vertex. If an adjacent has not been visited,
29             // mark it visited and enqueue it
30             for (int x : adj.get(curr)) {
31                 if (!visited[x]) {
32                     visited[x] = true;
33                     q.offer(x);
34                 }
```

```
35              }
36          }
37      }
38
39      // Function to add an edge to the graph
40      static void addEdge(List<List<Integer>> adj,
41                          int u, int v) {
42          adj.get(u).add(v);
43          adj.get(v).add(u);
44      }
45
46      public static void main(String[] args) {
47          // Number of vertices in the graph
48          int V = 5;
49
50          // Adjacency list representation of the graph
51          List<List<Integer>> adj = new ArrayList<>();
52          for (int i = 0; i < V; i++) {
53              adj.add(new ArrayList<>());
54          }
55
56          // Add edges to the graph
57          addEdge(adj, 0, 1);
58          addEdge(adj, 0, 2);
59          addEdge(adj, 1, 3);
60          addEdge(adj, 1, 4);
61          addEdge(adj, 2, 4);
62
63          // Mark all the vertices as not visited
64          boolean[] visited = new boolean[V];
65
66          // Perform BFS traversal starting from vertex 0
67          System.out.println("BFS starting from 0 : ");
68          bfs(adj, 0, visited);
69      }
70  }
```

Listing 2.1 – Implementação de BFS em Java

Autor: (GeeksforGeeks, 2021).

2.3.1 Explicação do Código

O código Java apresentado implementa a travessia de um grafo utilizando a busca em largura (BFS). A função ‘bfs‘ realiza a travessia a partir de um vértice inicial ‘s‘, marcando os vértices como visitados e enfileirando-os em uma fila para garantir que todos os vértices adjacentes sejam processados em ordem. A função ‘addEdge‘ é utilizada para adicionar arestas ao grafo representado por uma lista de adjacência. No método ‘main‘, um grafo com 5 vértices

é criado, arestas são adicionadas para definir a estrutura do grafo, e a função ‘bfs‘ é chamada para iniciar a travessia a partir do vértice 0, exibindo a ordem dos vértices visitados.

2.4 **Busca em Grafos Ponderados:**

Uma das desvantagens do BFS é que, embora encontre o caminho mais curto em grafos não ponderados, em grafos ponderados, ele não garante o caminho mais curto se os pesos das arestas forem diferentes. Para encontrar o caminho mais curto em grafos ponderados, algoritmos como Dijkstra ou Bellman-Ford são mais apropriados.

3 Quadtree

“A simplicidade é o último grau da sofisticação.”
Leonardo da Vinci

3.1 Definição

Quadtree é uma estrutura de dados hierárquica utilizada para dividir um espaço bidimensional em quatro sub-regiões menores, chamadas de quadrantes, nas quais o espaço é dividido recursivamente em quadrantes menores até que uma condição de parada seja atingida, como um número mínimo de pontos por região ou uma profundidade máxima na árvore. Essa estrutura é particularmente útil para armazenar e manipular dados espaciais, especialmente quando o espaço não é uniformemente distribuído. (SAMET, 1984).

3.2 Aplicação em Consultas Espaciais

Quadtrees são eficazes para outras consultas espaciais, como encontrar todos os pontos dentro de uma área retangular ou realizar operações de interseção e união de áreas. Essas operações são realizadas de maneira mais eficiente do que em estruturas de dados que não são adaptadas ao espaço bidimensional.

3.3 Complexidade da Quadtree

A Quadtree é uma estrutura de dados hierárquica utilizada para dividir um espaço bidimensional em quadrantes menores, facilitando a manipulação e busca de dados espaciais. A análise da complexidade dessa estrutura é crucial para entender seu desempenho em diferentes cenários. A seguir, discutem-se a complexidade de tempo para as operações de inserção e busca, bem como a complexidade de espaço.

3.3.1 Complexidade de Tempo

A complexidade de tempo para inserir um nó em uma Quadtree depende da profundidade da árvore e do número de subdivisões necessárias. Em um cenário ideal, onde a Quadtree está bem balanceada, a profundidade da árvore é $O(\log N)$, onde N é o número total de pontos. Nesse caso, a inserção pode ser realizada em tempo $O(\log N)$, uma vez que o nó é inserido no quadrante apropriado e pode exigir a divisão de quadrantes. No entanto, no pior caso, especialmente quando muitos pontos estão agrupados em regiões pequenas, a complexidade pode se aproximar de $O(\sqrt{N})$, refletindo uma árvore mais profunda e desbalanceada (SAMET, 1984).

Para a operação de busca, a complexidade é similar à de inserção. Em um cenário balanceado, a busca pode ser realizada em $O(\log N)$, pois o algoritmo percorre apenas os qua-

drantes relevantes. Contudo, em uma árvore desbalanceada, a complexidade da busca pode aumentar para $O(\sqrt{N})$, devido à profundidade aumentada (SEDGEWICK; WAYNE, 2011).

3.3.2 Complexidade de Espaço

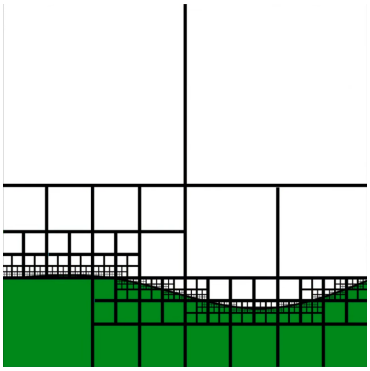
A complexidade de espaço de uma Quadtree é $O(N)$, onde N representa o número total de pontos armazenados. Esse espaço é necessário para armazenar tanto os pontos quanto os ponteiros para os quadrantes e nós. Em situações práticas, a árvore pode expandir-se para acomodar cada ponto em sua própria subdivisão, resultando em um uso de memória proporcional ao número de pontos armazenados (CORMEN et al., 2009).

3.3.3 Considerações Adicionais

A profundidade da Quadtree e seu balanceamento são fatores importantes que influenciam a complexidade das operações. Em um cenário ideal, a profundidade é $O(\log N)$, mas em cenários desbalanceados, a profundidade pode ser $O(\sqrt{N})$, impactando negativamente o desempenho das operações de inserção e busca. Técnicas de balanceamento são frequentemente aplicadas para manter a eficiência da Quadtree e evitar degradação no desempenho (SAMET, 1984).

3.4 Exemplo Prático: Detecção de Colisão em Jogos

Figura 3 – Quadtree em jogos



O Quadtree é chamado recursivamente se algum dos quadrantes gerados intersectam o terreno (em verde), até atingir a condição de parada estipulada. Imagem proveniente: (MRHEYHEYHEY27, 2014)

Um dos exemplos práticos mais comuns do uso de Quadtrees é na detecção de colisões em jogos. O processo de detecção de colisão pode ser descrito da seguinte forma:

1. **Definição do Terreno**: Inicialmente, define-se o terreno ou área de jogo.
2. **Divisão Inicial**: Sobrepõe-se um quadrado que cobre todo o terreno, criando o nó raiz da Quadtree.

3. **Divisão Recursiva**: A raiz é dividida em quatro quadrantes, criando os nós filhos. Esse processo continua até que os quadrantes sejam suficientemente pequenos ou atendam a uma condição de parada.

4. **Classificação dos Quadrantes**: Cada quadrante é classificado como "vazio"ou "sólido"com base na presença de objetos, facilitando a detecção de colisões futuras.

3.5 Codificação

Segue um exemplo de implementação de uma Quadtree em Java:

```
class Point {
    int x, y;

    Point(int _x, int _y) {
        x = _x;
        y = _y;
    }

    Point() {
        x = 0;
        y = 0;
    }
}

class Node {
    Point pos;
    int data;

    Node(Point _pos, int _data) {
        pos = _pos;
        data = _data;
    }

    Node() {
        data = 0;
    }
}

public class Quad {
    Point topLeft;
    Point botRight;
    Node n;
    Quad topLeftTree;
    Quad topRightTree;
    Quad botLeftTree;
    Quad botRightTree;

```

```
38      // Default constructor
39      public Quad() {
40          topLeft = new Point(0, 0);
41          botRight = new Point(0, 0);
42          n = null;
43          topLeftTree = null;
44          topRightTree = null;
45          botLeftTree = null;
46          botRightTree = null;
47      }
48
49      // Parameterized constructor
50      public Quad(Point topL, Point botR) {
51          n = null;
52          topLeftTree = null;
53          topRightTree = null;
54          botLeftTree = null;
55          botRightTree = null;
56          topLeft = topL;
57          botRight = botR;
58      }
59
60      // Insert a node into the quadtree
61      void insert(Node node) {
62          if (node == null)
63              return;
64
65          if (!inBoundary(node.pos))
66              return;
67
68          if (Math.abs(topLeft.x - botRight.x) <= 1 && Math.abs(topLeft.y
                - botRight.y) <= 1) {
69              // We are at a quad of unit area; cannot subdivide further
70              if (n == null)
71                  n = node;
72              return;
73          }
74
75          if ((topLeft.x + botRight.x) / 2 >= node.pos.x) {
76              if ((topLeft.y + botRight.y) / 2 >= node.pos.y) {
77                  // Indicates topLeftTree
78                  if (topLeftTree == null)
79                      topLeftTree = new Quad(
80                              new Point(topLeft.x, topLeft.y),
81                              new Point((topLeft.x + botRight.x) / 2, (
                                    topLeft.y + botRight.y) / 2));
82                  topLeftTree.insert(node);
83              } else {
```

```
84                  // Indicates botLeftTree
85                  if (botLeftTree == null)
86                      botLeftTree = new Quad(
87                              new Point(topLeft.x, (topLeft.y + botRight.y
                                    ) / 2),
88                              new Point((topLeft.x + botRight.x) / 2,
                                    botRight.y));
89                  botLeftTree.insert(node);
90              }
91          } else {
92              if ((topLeft.y + botRight.y) / 2 >= node.pos.y) {
93                  // Indicates topRightTree
94                  if (topRightTree == null)
95                      topRightTree = new Quad(
96                              new Point((topLeft.x + botRight.x) / 2,
                                    topLeft.y),
97                              new Point(botRight.x, (topLeft.y + botRight.
                                    y) / 2));
98                  topRightTree.insert(node);
99              } else {
100                 // Indicates botRightTree
101                 if (botRightTree == null)
102                     botRightTree = new Quad(
103                             new Point((topLeft.x + botRight.x) / 2, (
                                    topLeft.y + botRight.y) / 2),
104                             new Point(botRight.x, botRight.y));
105                 botRightTree.insert(node);
106             }
107         }
108     }
109
110     // Search for a node in the quadtree
111     Node search(Point p) {
112         if (!inBoundary(p))
113             return null;
114
115         if (n != null)
116             return n;
117
118         if ((topLeft.x + botRight.x) / 2 >= p.x) {
119             if ((topLeft.y + botRight.y) / 2 >= p.y) {
120                 // Indicates topLeftTree
121                 if (topLeftTree == null)
122                     return null;
123                 return topLeftTree.search(p);
124             } else {
125                 // Indicates botLeftTree
126                 if (botLeftTree == null)
```

```
127                     return null;
128                 return botLeftTree.search(p);
129             }
130         } else {
131             if ((topLeft.y + botRight.y) / 2 >= p.y) {
132                 // Indicates topRightTree
133                 if (topRightTree == null)
134                     return null;
135                 return topRightTree.search(p);
136             } else {
137                 // Indicates botRightTree
138                 if (botRightTree == null)
139                     return null;
140                 return botRightTree.search(p);
141             }
142         }
143     }
144 
145     // Check if a point is within the boundary of the quadtree
146     boolean inBoundary(Point p) {
147         return (p.x >= topLeft.x && p.x <= botRight.x && p.y >= topLeft.
                y && p.y <= botRight.y);
148     }
149 
150     // Driver program
151     public static void main(String[] args) {
152         Quad center = new Quad(new Point(0, 0), new Point(8, 8));
153         Node a = new Node(new Point(1, 1), 1);
154         Node b = new Node(new Point(2, 5), 2);
155         Node c = new Node(new Point(7, 6), 3);
156         center.insert(a);
157         center.insert(b);
158         center.insert(c);
159         System.out.println("Node a: " + center.search(new Point(1, 1)).
                data);
160         System.out.println("Node b: " + center.search(new Point(2, 5)).
                data);
161         System.out.println("Node c: " + center.search(new Point(7, 6)).
                data);
162         System.out.println("Non-existing node: " + center.search(new
                Point(5, 5)));
163     }
164 }
```

Listing 3.1 – Implementação de uma Quadtree em Java

Autor: (GUPTA, 2021).

3.5.1 Explicação do Código

O código em Java implementa uma Quadtree. A classe ‘Point‘ representa um ponto com coordenadas x e y, enquanto a classe ‘Node‘ armazena um ponto e um dado associado. A classe ‘Quad‘ representa um quadrante na Quadtree, com métodos para inserir nós e buscar nós dentro do quadrante. O método ‘insert‘ subdivide o quadrante em quatro quadrantes menores conforme necessário e insere o nó no quadrante apropriado. O método ‘search‘ localiza um nó verificando se está dentro dos limites do quadrante e navegando pelos quadrantes filhos. No ‘main‘, o código cria uma Quadtree, adiciona alguns nós e realiza buscas para verificar a presença desses nós.

3.6 Vantagem e Desvantagem

3.6.1 Espaços não uniformes

Quadtrees são particularmente vantajosas em cenários onde a distribuição de dados é irregular ou não uniforme. Em áreas com alta densidade de dados, os quadrantes se subdividem mais rapidamente, enquanto em áreas de baixa densidade, a subdivisão é menos intensa, o que permite uma utilização mais eficiente da memória e um desempenho melhorado.

3.6.2 Inserção e Remoção

A principal desvantagem da Quadtree vem a ser o custo de inserção e remoção que pode ser elevado em situações extremas, particularmente quando o número de subdivisões é grande.

Isso ocorre porque, à medida que o espaço é subdividido em quadrantes cada vez menores, a estrutura pode se tornar mais profunda e complexa. Com muitas subdivisões, a inserção e a remoção de elementos exigem a navegação por múltiplos níveis da árvore e a reorganização dos quadrantes, o que pode aumentar o tempo de processamento e o custo computacional.

Além disso, o gerenciamento de muitos quadrantes pode levar a um uso mais intenso de memória e a um desempenho reduzido, especialmente em cenários dinâmicos onde as operações de inserção e remoção são frequentes.

Referências

BARBOSA, D. *ArvoreBinariaApp*. 2017. Disponível em: <"https://gist.github.com/divanibarbosa/819e7cfcf1b9bae48c4e0f5bd74fb658.js">. Citado na página 17.

CORMEN, T. H. et al. *Introduction to Algorithms*. 3rd. ed. [S.l.]: MIT Press, 2009. Citado nas páginas 21, 22 e 26.

GeeksforGeeks. *Breadth First Search or BFS for a Graph*. 2021. Acessado em: 24-ago-2024. Disponível em: <https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/>. Citado na página 23.

GOODRICH, M. T.; TAMASSIA, R. *Estrutura de Dados e Algoritmos em Java*. [S.l.]: Bookman, 2013. Citado na página 10.

GRONER, L. *Estrutura de Dados e Algoritmos com JavaScript*. [S.l.]: NovaTec, 2019. Citado na página 11.

GUPTA, S. *Quad Tree*. 2021. Acessado em: 24-ago-2024. Disponível em: <https://www.geeksforgeeks.org/quad-tree/>. Citado na página 30.

MRHEYHEYHEY27. *Quadtree Explanation*. 2014. Acessado em: 24-ago-2024. Disponível em: <https://www.youtube.com/watch?v=jxbDYxm-pXg>. Citado na página 26.

RICARDO. *Estrutura de Dados*. [S.l.]: SESES, 2015. Citado na página 10.

SAMBOL, M. *Breadth-first search in 4 minutes*. 2022. Accessed: 2024-08-23. Disponível em: <https://www.youtube.com/watch?v=HZ5YTanv5QE>. Citado na página 21.

SAMET, H. The quadtree and related hierarchical data structures. *ACM Computing Surveys (CSUR)*, ACM, v. 16, n. 2, p. 187–260, 1984. Citado nas páginas 25 e 26.

SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4th. ed. [S.l.]: Addison-Wesley, 2011. Citado nas páginas 21, 22 e 26.

VATSA, K. *Inorder Tree Traversal without Recursion*. 2023. Disponível em: <https://www.geeksforgeeks.org/inorder-tree-traversal-without-recursion/>. Acesso em: 9 ago 2023. Citado na página 17.